

Experiment ‘p_14_2_selection_sort_interval_array’ Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 3

Problem Specification

Problem name: p_14_2_selection_sort_interval_array

Natural language statement: Write a method that implements the selection sort algorithm to sort an array of values from a finite interval.

Method signature: p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted: array<int>)

Ensures

- forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
- multiset(inputs[..]) == multiset(sorted[..])

Requires

- forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that implements the selection sort algorithm to sort an array of values
↪ from a finite interval.

The signature should be:

p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted:
↪ array<int>)

The method should respect the following contract:

requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound, ensures forall i
↪ , j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures multiset(
↪ inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKLtbjOdaDrHLkBRhRlPJqUee9Qw

Model: gpt-5.5-2026-04-23

Created at: 1780648677

Finish reason: stop

Usage: 380 tokens in, and 8948 tokens out

```
{
  sorted := new int[inputs.Length];

  var c := 0;
  while c < inputs.Length
    invariant 0 <= c <= inputs.Length
    invariant sorted.Length == inputs.Length
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }

  forall k: int | 0 <= k < inputs.Length
    ensures sorted[..][k] == inputs[..][k]
  {
    assert sorted[k] == inputs[k];
  }
}
```

```

}
assert sorted[..] == inputs[..];

var n := sorted.Length;
var i := 0;
while i < n
  invariant sorted.Length == n
  invariant inputs.Length == n
  invariant 0 <= i <= n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
  invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
{
  var min := i;
  var j := i + 1;

  while j < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant i < n
    invariant 0 <= i < j <= n
    invariant i <= min < j
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
    invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
    invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      min := j;
      forall k: int | i <= k < j
        ensures sorted[min] <= sorted[k]
      {
        assert sorted[oldMin] <= sorted[k];
        assert sorted[min] < sorted[oldMin];
      }
    }
  }

  forall k: int | i <= k < j + 1
    ensures sorted[min] <= sorted[k]
  {
    if k < j {
      assert sorted[min] <= sorted[k];
    } else {
      assert k == j;
      assert sorted[min] <= sorted[j];
    }
  }
  j := j + 1;
}

```

```

ghost var pre := sorted[..];
ghost var min0 := min;
ghost var preIVal := pre[i];
ghost var preMinVal := pre[min0];

assert i <= min0 < n;
assert forall k, l :: 0 <= k < l < i ==> pre[k] <= pre[l];
assert forall k, l :: 0 <= k < i <= l < n ==> pre[k] <= pre[l];
assert forall k :: i <= k < n ==> preMinVal <= pre[k];

if min0 != i {
  assert i < min0 < n;
  var tmp := sorted[i];
  assert tmp == preIVal;
  assert sorted[min0] == preMinVal;
  sorted[i] := sorted[min0];
  sorted[min0] := tmp;

  assert sorted[i] == preMinVal;
  assert sorted[min0] == preIVal;
  assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];

  ghost var post := sorted[..];
  assert post == pre[i := preMinVal][min0 := preIVal];

  ghost var left := pre[..i];
  ghost var middle := pre[i + 1..min0];
  ghost var right := pre[min0 + 1..];
  assert pre == left + [preIVal] + middle + [preMinVal] + right;
  assert post == left + [preMinVal] + middle + [preIVal] + right;

  calc {
    multiset(post);
  ==
    multiset(left + [preMinVal] + middle + [preIVal] + right);
  ==
    multiset(left) + multiset([preMinVal]) + multiset(middle) + multiset([preIVal]) +
    ↪ multiset(right);
  ==
    multiset(left) + multiset([preIVal]) + multiset(middle) + multiset([preMinVal]) +
    ↪ multiset(right);
  ==
    multiset(left + [preIVal] + middle + [preMinVal] + right);
  ==
    multiset(pre);
  }
  assert multiset(sorted[..]) == multiset(pre);
} else {
  assert sorted[..] == pre;
  assert multiset(sorted[..]) == multiset(pre);
}

```

```

assert sorted[i] == preMinVal;
assert sorted[min0] == preIVal;
assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];
assert multiset(sorted[..]) == multiset(inputs[..]);

forall k: int, l: int | 0 <= k < l < i + 1
  ensures sorted[k] <= sorted[l]
{
  if l < i {
    assert k != i && l != i;
    assert k != min0 && l != min0;
    assert sorted[k] == pre[k];
    assert sorted[l] == pre[l];
    assert pre[k] <= pre[l];
  } else {
    assert l == i;
    assert k < i;
    assert k != min0;
    assert sorted[k] == pre[k];
    assert sorted[i] == preMinVal;
    assert pre[k] <= pre[min0];
  }
}

forall k: int, l: int | 0 <= k < i + 1 <= l < n
  ensures sorted[k] <= sorted[l]
{
  if k < i {
    assert k != i && k != min0;
    assert sorted[k] == pre[k];
    if l == min0 {
      assert sorted[l] == preIVal;
      assert pre[k] <= pre[i];
    } else {
      assert l != i;
      assert sorted[l] == pre[l];
      assert pre[k] <= pre[l];
    }
  } else {
    assert k == i;
    assert sorted[k] == preMinVal;
    if l == min0 {
      assert sorted[l] == preIVal;
      assert preMinVal <= pre[i];
    } else {
      assert l != i;
      assert sorted[l] == pre[l];
      assert preMinVal <= pre[l];
    }
  }
}

```

```

    }

    i := i + 1;
}

assert i == n;
assert n == sorted.Length;
assert forall k, l :: 0 <= k < l < sorted.Length ==> sorted[k] <= sorted[l];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_14_2_selection_sort_interval_array/program.dfy(88,16): Error: assignment
  ↳ to array element is not allowed in this context, because the statement is in a
  ↳ ghost context; e.g., it may be guarded by a specification-only expression
|

```

```

88 | sorted[i] := sorted[min0];
| ^^

```

```

problem_data/p_14_2_selection_sort_interval_array/program.dfy(89,19): Error: assignment
  ↳ to array element is not allowed in this context, because the statement is in a
  ↳ ghost context; e.g., it may be guarded by a specification-only expression
|

```

```

89 | sorted[min0] := tmp;
| ^^

```

2 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

- ↳ Dafny body, including the curly braces that surround it. Do not show the signature
- ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKOrPhveeFuKtm4pX2wmioOE7tbL

Model: gpt-5.5-2026-04-23

Created at: 1780648861

Finish reason: stop

Usage: 2351 tokens in, and 2242 tokens out

```

{
    sorted := new int[inputs.Length];

    var c := 0;
    while c < inputs.Length
        invariant 0 <= c <= inputs.Length
        invariant sorted.Length == inputs.Length

```

```

    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }

forall k: int | 0 <= k < inputs.Length
  ensures sorted[..][k] == inputs[..][k]
{
  assert sorted[k] == inputs[k];
}
assert sorted[..] == inputs[..];

var n := sorted.Length;
var i := 0;
while i < n
  invariant sorted.Length == n
  invariant inputs.Length == n
  invariant 0 <= i <= n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
  invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
{
  var min := i;
  var j := i + 1;

  while j < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant i < n
    invariant 0 <= i < j <= n
    invariant i <= min < j
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
    invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
    invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      min := j;
      forall k: int | i <= k < j
        ensures sorted[min] <= sorted[k]
      {
        assert sorted[oldMin] <= sorted[k];
        assert sorted[min] < sorted[oldMin];
      }
    }
  }

  forall k: int | i <= k < j + 1
    ensures sorted[min] <= sorted[k]
  {

```

```

    if k < j {
      assert sorted[min] <= sorted[k];
    } else {
      assert k == j;
      assert sorted[min] <= sorted[j];
    }
  }
  j := j + 1;
}

ghost var pre := sorted[..];
ghost var min0 := min;
ghost var preIVal := pre[i];
ghost var preMinVal := pre[min0];

assert i <= min0 < n;
assert forall k, l :: 0 <= k < l < i ==> pre[k] <= pre[l];
assert forall k, l :: 0 <= k < i <= l < n ==> pre[k] <= pre[l];
assert forall k :: i <= k < n ==> preMinVal <= pre[k];

if min != i {
  assert min == min0;
  assert i < min < n;
  assert i < min0 < n;
  var tmp := sorted[i];
  assert tmp == preIVal;
  assert sorted[min] == preMinVal;
  sorted[i] := sorted[min];
  sorted[min] := tmp;

  assert sorted[i] == preMinVal;
  assert sorted[min0] == preIVal;
  assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];

  ghost var post := sorted[..];
  assert post == pre[i := preMinVal][min0 := preIVal];

  ghost var left := pre[..i];
  ghost var middle := pre[i + 1..min0];
  ghost var right := pre[min0 + 1..];
  assert pre == left + [preIVal] + middle + [preMinVal] + right;
  assert post == left + [preMinVal] + middle + [preIVal] + right;

  calc {
    multiset(post);
  ==
    multiset(left + [preMinVal] + middle + [preIVal] + right);
  ==
    multiset(left) + multiset([preMinVal]) + multiset(middle) + multiset([preIVal]) +
    ↪ multiset(right);
  ==

```



```

    multiset(left) + multiset([preIVal]) + multiset(middle) + multiset([preMinVal]) +
    ↪ multiset(right);
==
    multiset(left + [preIVal] + middle + [preMinVal] + right);
==
    multiset(pre);
}
assert multiset(sorted[..]) == multiset(pre);
} else {
    assert min == i;
    assert min0 == i;
    assert sorted[..] == pre;
    assert multiset(sorted[..]) == multiset(pre);
}

assert sorted[i] == preMinVal;
assert sorted[min0] == preIVal;
assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];
assert multiset(sorted[..]) == multiset(inputs[..]);

forall k: int, l: int | 0 <= k < l < i + 1
    ensures sorted[k] <= sorted[l]
{
    if l < i {
        assert k != i && l != i;
        assert k != min0 && l != min0;
        assert sorted[k] == pre[k];
        assert sorted[l] == pre[l];
        assert pre[k] <= pre[l];
    } else {
        assert l == i;
        assert k < i;
        assert k != min0;
        assert sorted[k] == pre[k];
        assert sorted[i] == preMinVal;
        assert pre[k] <= pre[min0];
    }
}

forall k: int, l: int | 0 <= k < i + 1 <= l < n
    ensures sorted[k] <= sorted[l]
{
    if k < i {
        assert k != i && k != min0;
        assert sorted[k] == pre[k];
        if l == min0 {
            assert sorted[l] == preIVal;
            assert pre[k] <= pre[i];
        } else {
            assert l != i;
            assert sorted[l] == pre[l];

```

```

        assert pre[k] <= pre[l];
    }
} else {
    assert k == i;
    assert sorted[k] == preMinVal;
    if l == min0 {
        assert sorted[l] == preIVal;
        assert preMinVal <= pre[i];
    } else {
        assert l != i;
        assert sorted[l] == pre[l];
        assert preMinVal <= pre[l];
    }
}
}

i := i + 1;
}

assert i == n;
assert n == sorted.Length;
assert forall k, l :: 0 <= k < l < sorted.Length ==> sorted[k] <= sorted[l];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_2_selection_sort_interval_array/program.dfy(1,7): Error: Verification
  ↳ of 'p_14_2_selection_sort_interval_array' timed out after 30 seconds. (the limit
  ↳ can be increased using --verification-time-limit)
|
1 | method p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (
  ↳ sorted: array<int>)
| ~~~~~

```

Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKQIAAVVnv83J9JRyBwJb7g0Ex5K

Model: gpt-5.5-2026-04-23

Created at: 1780648950

Finish reason: stop

Usage: 4228 tokens in, and 6291 tokens out

```
{
  sorted := new int[inputs.Length];

  var c := 0;
  while c < inputs.Length
    invariant 0 <= c <= inputs.Length
    invariant sorted.Length == inputs.Length
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
    {
      sorted[c] := inputs[c];
      c := c + 1;
    }

  forall k: int | 0 <= k < inputs.Length
    ensures sorted[..][k] == inputs[..][k]
  {
    assert sorted[k] == inputs[k];
  }
  assert sorted[..] == inputs[..];
  assert {:split_here} true;

  var n := sorted.Length;
  var i := 0;
  while i < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i <= n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
    invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
    {
      var min := i;
      var j := i + 1;

      while j < n
        invariant sorted.Length == n
        invariant inputs.Length == n
        invariant i < n
        invariant 0 <= i < j <= n
        invariant i <= min < j
        invariant multiset(sorted[..]) == multiset(inputs[..])
        invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
        invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
        invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
        {
          if sorted[j] < sorted[min] {
            var oldMin := min;
            min := j;
            forall k: int | i <= k < j
```

```

    ensures sorted[min] <= sorted[k]
  {
    assert sorted[oldMin] <= sorted[k];
    assert sorted[min] < sorted[oldMin];
  }
}

forall k: int | i <= k < j + 1
  ensures sorted[min] <= sorted[k]
{
  if k < j {
    assert sorted[min] <= sorted[k];
  } else {
    assert k == j;
    assert sorted[min] <= sorted[j];
  }
}
j := j + 1;
}

assert {:split_here} true;

ghost var pre := sorted[..];
ghost var min0 := min;
ghost var preIVal := pre[i];
ghost var preMinVal := pre[min0];

assert i <= min0 < n;
assert forall k, l :: 0 <= k < l < i ==> pre[k] <= pre[l];
assert forall k, l :: 0 <= k < i <= l < n ==> pre[k] <= pre[l];
assert forall k :: i <= k < n ==> preMinVal <= pre[k];

if min != i {
  assert min == min0;
  assert i < min < n;
  assert i < min0 < n;
  var tmp := sorted[i];
  assert tmp == preIVal;
  assert sorted[min] == preMinVal;
  sorted[i] := sorted[min];
  sorted[min] := tmp;

  assert sorted[i] == preMinVal;
  assert sorted[min0] == preIVal;
  assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];

  ghost var post := sorted[..];
  assert post == pre[i := preMinVal][min0 := preIVal];

  ghost var left := pre[..i];
  ghost var middle := pre[i + 1..min0];

```

```

ghost var right := pre[min0 + 1..];
assert pre == left + [preIVal] + middle + [preMinVal] + right;
assert post == left + [preMinVal] + middle + [preIVal] + right;

calc {
  multiset(post);
==
  multiset(left + [preMinVal] + middle + [preIVal] + right);
==
  multiset(left) + multiset([preMinVal]) + multiset(middle) + multiset([preIVal]) +
    ↪ multiset(right);
==
  multiset(left) + multiset([preIVal]) + multiset(middle) + multiset([preMinVal]) +
    ↪ multiset(right);
==
  multiset(left + [preIVal] + middle + [preMinVal] + right);
==
  multiset(pre);
}
assert multiset(sorted[..]) == multiset(pre);
} else {
  assert min == i;
  assert min0 == i;
  assert sorted[..] == pre;
  assert multiset(sorted[..]) == multiset(pre);
}

assert {:split_here} true;

assert sorted[i] == preMinVal;
assert sorted[min0] == preIVal;
assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];
assert multiset(sorted[..]) == multiset(inputs[..]);

forall k: int, l: int | 0 <= k < l < i + 1
  ensures sorted[k] <= sorted[l]
{
  if l < i {
    assert k != i && l != i;
    assert k != min0 && l != min0;
    assert sorted[k] == pre[k];
    assert sorted[l] == pre[l];
    assert pre[k] <= pre[l];
  } else {
    assert l == i;
    assert k < i;
    assert k != min0;
    assert sorted[k] == pre[k];
    assert sorted[i] == preMinVal;
    assert pre[k] <= pre[min0];
  }
}

```

```

}

assert {:split_here} true;

forall k: int, l: int | 0 <= k < i + 1 <= l < n
  ensures sorted[k] <= sorted[l]
{
  if k < i {
    assert k != i && k != min0;
    assert sorted[k] == pre[k];
    if l == min0 {
      assert sorted[l] == pre[l];
      assert pre[k] <= pre[l];
    } else {
      assert l != i;
      assert sorted[l] == pre[l];
      assert pre[k] <= pre[l];
    }
  } else {
    assert k == i;
    assert sorted[k] == preMinVal;
    if l == min0 {
      assert sorted[l] == pre[l];
      assert preMinVal <= pre[l];
    } else {
      assert l != i;
      assert sorted[l] == pre[l];
      assert preMinVal <= pre[l];
    }
  }
}

i := i + 1;
}

assert i == n;
assert n == sorted.Length;
assert forall k, l :: 0 <= k < l < sorted.Length ==> sorted[k] <= sorted[l];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_2_selection_sort_interval_array/program.dfy(74,4): Error: Verification
  ↳ of 'p_14_2_selection_sort_interval_array' timed out after 32 seconds. (the limit
  ↳ can be increased using --verification-time-limit)
|
74 | assert {:split_here} true;
|   ~~~~~

```

Dafny program verifier finished with 5 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKTcRJF4nSQtg4pvtvjFen4SwKG7

Model: gpt-5.5-2026-04-23

Created at: 1780649156

Finish reason: stop

Usage: 6137 tokens in, and 4450 tokens out

```
{
  sorted := new int[inputs.Length];

  var c := 0;
  while c < inputs.Length
    invariant 0 <= c <= inputs.Length
    invariant sorted.Length == inputs.Length
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }

  forall k: int | 0 <= k < inputs.Length
    ensures sorted[..][k] == inputs[..][k]
  {
    assert sorted[k] == inputs[k];
  }
  assert sorted[..] == inputs[..];

  var n := sorted.Length;
  var i := 0;
  while i < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i <= n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
    invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
  {
    var min := i;
    var j := i + 1;

    while j < n
```

```

invariant sorted.Length == n
invariant 0 <= i < j <= n
invariant i <= min < j
invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
{
  if sorted[j] < sorted[min] {
    var oldMin := min;
    min := j;
    forall k: int | i <= k < j
      ensures sorted[min] <= sorted[k]
    {
      assert sorted[oldMin] <= sorted[k];
      assert sorted[min] < sorted[oldMin];
    }
  }
}

forall k: int | i <= k < j + 1
  ensures sorted[min] <= sorted[k]
{
  if k < j {
    assert sorted[min] <= sorted[k];
  } else {
    assert k == j;
    assert sorted[min] <= sorted[j];
  }
}
j := j + 1;
}

ghost var pre := sorted[..];
ghost var min0 := min;
ghost var preIVal := pre[i];
ghost var preMinVal := pre[min0];

assert i <= min0 < n;
assert forall k, l :: 0 <= k < l < i ==> pre[k] <= pre[l];
assert forall k, l :: 0 <= k < i <= l < n ==> pre[k] <= pre[l];
assert forall k :: i <= k < n ==> preMinVal <= pre[k];

if min != i {
  assert min == min0;
  assert i < min0 < n;
  var tmp := sorted[i];
  sorted[i] := sorted[min];
  sorted[min] := tmp;

  assert sorted[i] == preMinVal;
  assert sorted[min0] == preIVal;
  assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];

  ghost var post := sorted[..];

```



```

assert post == pre[i := preMinVal][min0 := preIVal];

ghost var left := pre[..i];
ghost var middle := pre[i + 1..min0];
ghost var right := pre[min0 + 1..];
assert pre == left + [preIVal] + middle + [preMinVal] + right;
assert post == left + [preMinVal] + middle + [preIVal] + right;

calc {
  multiset(post);
==
  multiset(left + [preMinVal] + middle + [preIVal] + right);
==
  multiset(left) + multiset([preMinVal]) + multiset(middle) + multiset([preIVal]) +
    ↪ multiset(right);
==
  multiset(left) + multiset([preIVal]) + multiset(middle) + multiset([preMinVal]) +
    ↪ multiset(right);
==
  multiset(left + [preIVal] + middle + [preMinVal] + right);
==
  multiset(pre);
}
assert multiset(sorted[..]) == multiset(pre);
} else {
  assert min0 == i;
  assert sorted[..] == pre;
  assert multiset(sorted[..]) == multiset(pre);
}

assert sorted[i] == preMinVal;
assert sorted[min0] == preIVal;
assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];
assert multiset(sorted[..]) == multiset(inputs[..]);

forall k: int, l: int | 0 <= k < l < i + 1
  ensures sorted[k] <= sorted[l]
{
  if l < i {
    assert k != i && l != i;
    assert k != min0 && l != min0;
    assert sorted[k] == pre[k];
    assert sorted[l] == pre[l];
    assert pre[k] <= pre[l];
  } else {
    assert l == i;
    assert k < i;
    assert k != min0;
    assert sorted[k] == pre[k];
    assert sorted[i] == preMinVal;
    assert pre[k] <= pre[min0];
  }
}

```

```

    }
  }

  forall k: int, l: int | 0 <= k < i + 1 <= l < n
    ensures sorted[k] <= sorted[l]
  {
    if k < i {
      assert k != i && k != min0;
      assert sorted[k] == pre[k];
      if l == min0 {
        assert min0 != i;
        assert sorted[l] == pre[l];
        assert pre[k] <= pre[i];
      } else {
        assert l != i;
        assert sorted[l] == pre[l];
        assert pre[k] <= pre[l];
      }
    } else {
      assert k == i;
      assert sorted[k] == preMinVal;
      if l == min0 {
        assert min0 != i;
        assert sorted[l] == pre[l];
        assert preMinVal <= pre[i];
      } else {
        assert l != i;
        assert sorted[l] == pre[l];
        assert preMinVal <= pre[l];
      }
    }
  }

  i := i + 1;
}

assert i == n;
assert n == sorted.Length;
assert forall k, l :: 0 <= k < l < sorted.Length ==> sorted[k] <= sorted[l];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Final Program

```

method p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (
  ↪ sorted: array<int>)
  requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound
  ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
  ensures multiset(inputs[..]) == multiset(sorted[..])
{

```

```

sorted := new int[inputs.Length];

var c := 0;
while c < inputs.Length
  invariant 0 <= c <= inputs.Length
  invariant sorted.Length == inputs.Length
  invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
{
  sorted[c] := inputs[c];
  c := c + 1;
}

forall k: int | 0 <= k < inputs.Length
  ensures sorted[..][k] == inputs[..][k]
{
  assert sorted[k] == inputs[k];
}
assert sorted[..] == inputs[..];

var n := sorted.Length;
var i := 0;
while i < n
  invariant sorted.Length == n
  invariant inputs.Length == n
  invariant 0 <= i <= n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall k, l :: 0 <= k < l < i ==> sorted[k] <= sorted[l]
  invariant forall k, l :: 0 <= k < i <= l < n ==> sorted[k] <= sorted[l]
{
  var min := i;
  var j := i + 1;

  while j < n
    invariant sorted.Length == n
    invariant 0 <= i < j <= n
    invariant i <= min < j
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      min := j;
      forall k: int | i <= k < j
        ensures sorted[min] <= sorted[k]
      {
        assert sorted[oldMin] <= sorted[k];
        assert sorted[min] < sorted[oldMin];
      }
    }
  }

  forall k: int | i <= k < j + 1
    ensures sorted[min] <= sorted[k]

```

```

{
  if k < j {
    assert sorted[min] <= sorted[k];
  } else {
    assert k == j;
    assert sorted[min] <= sorted[j];
  }
}
j := j + 1;
}

ghost var pre := sorted[..];
ghost var min0 := min;
ghost var preIVal := pre[i];
ghost var preMinVal := pre[min0];

assert i <= min0 < n;
assert forall k, l :: 0 <= k < l < i ==> pre[k] <= pre[l];
assert forall k, l :: 0 <= k < i <= l < n ==> pre[k] <= pre[l];
assert forall k :: i <= k < n ==> preMinVal <= pre[k];

if min != i {
  assert min == min0;
  assert i < min0 < n;
  var tmp := sorted[i];
  sorted[i] := sorted[min];
  sorted[min] := tmp;

  assert sorted[i] == preMinVal;
  assert sorted[min0] == preIVal;
  assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];

  ghost var post := sorted[..];
  assert post == pre[i := preMinVal][min0 := preIVal];

  ghost var left := pre[..i];
  ghost var middle := pre[i + 1..min0];
  ghost var right := pre[min0 + 1..];
  assert pre == left + [preIVal] + middle + [preMinVal] + right;
  assert post == left + [preMinVal] + middle + [preIVal] + right;

  calc {
    multiset(post);
  ==
    multiset(left + [preMinVal] + middle + [preIVal] + right);
  ==
    multiset(left) + multiset([preMinVal]) + multiset(middle) + multiset([preIVal]) +
    ↪ multiset(right);
  ==
    multiset(left) + multiset([preIVal]) + multiset(middle) + multiset([preMinVal]) +
    ↪ multiset(right);

```

```

    ==
    multiset(left + [preIVal] + middle + [preMinVal] + right);
    ==
    multiset(pre);
  }
  assert multiset(sorted[..]) == multiset(pre);
} else {
  assert min0 == i;
  assert sorted[..] == pre;
  assert multiset(sorted[..]) == multiset(pre);
}

assert sorted[i] == preMinVal;
assert sorted[min0] == preIVal;
assert forall t :: 0 <= t < n && t != i && t != min0 ==> sorted[t] == pre[t];
assert multiset(sorted[..]) == multiset(inputs[..]);

forall k: int, l: int | 0 <= k < l < i + 1
  ensures sorted[k] <= sorted[l]
{
  if l < i {
    assert k != i && l != i;
    assert k != min0 && l != min0;
    assert sorted[k] == pre[k];
    assert sorted[l] == pre[l];
    assert pre[k] <= pre[l];
  } else {
    assert l == i;
    assert k < i;
    assert k != min0;
    assert sorted[k] == pre[k];
    assert sorted[i] == preMinVal;
    assert pre[k] <= pre[min0];
  }
}

forall k: int, l: int | 0 <= k < i + 1 <= l < n
  ensures sorted[k] <= sorted[l]
{
  if k < i {
    assert k != i && k != min0;
    assert sorted[k] == pre[k];
    if l == min0 {
      assert min0 != i;
      assert sorted[l] == preIVal;
      assert pre[k] <= pre[i];
    } else {
      assert l != i;
      assert sorted[l] == pre[l];
      assert pre[k] <= pre[l];
    }
  }
}

```

```

    } else {
        assert k == i;
        assert sorted[k] == preMinVal;
        if l == min0 {
            assert min0 != i;
            assert sorted[l] == preIVal;
            assert preMinVal <= pre[i];
        } else {
            assert l != i;
            assert sorted[l] == pre[l];
            assert preMinVal <= pre[l];
        }
    }
}

i := i + 1;
}

assert i == n;
assert n == sorted.Length;
assert forall k, l :: 0 <= k < l < sorted.Length ==> sorted[k] <= sorted[l];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Total Token Usage

Input tokens: 13096

Output tokens: 21931

Reasoning tokens: 14940

Sum of ‘total tokens’: 35027

Experiment Timings

Overall Experiment started at 1780648676908, ended at 1780649304281, lasting 627373ms (627.37 seconds)

Iteration #4 started at 1780649155048, ended at 1780649304181, lasting 149133ms (149.13 seconds)

Iteration #1 started at 1780648676908, ended at 1780648861169, lasting 184261ms (184.26 seconds)

Iteration #2 started at 1780648861171, ended at 1780648949547, lasting 88376ms (88.38 seconds)

Iteration #3 started at 1780648949548, ended at 1780649154973, lasting 205425ms (205.43 seconds)